

Valores padrão e argumentos nomeados

Objetivo da Aula

- Entender a diferença entre parâmetro e argumento
 - Definir valores padrão para tornar parâmetros opcionais
 - Conhecer a regra de posição dos valores padrão
 - Usar argumentos nomeados (keyword arguments) para maior controle
 - Combinar argumentos posicionais e nomeados
 - Evitar a armadilha de valores padrão mutáveis
-

1. Parâmetro vs Argumento

Antes de avançar, vale esclarecer uma confusão comum de terminologia:

- **Parâmetro** é a variável definida na criação da função (entre os parênteses do `def`)
- **Argumento** é o valor concreto passado na chamada da função

```
def verificar_status(codigo): # "codigo" é o PARÂMETRO
    return 200 <= codigo < 300

resultado = verificar_status(200) # 200 é o ARGUMENTO
```

Na prática, os termos são usados como sinônimos e tudo bem. Mas saber a diferença ajuda a ler documentações de bibliotecas como `pytest` e `requests`, que falam em "keyword arguments" e "default parameter values".

2. Valores Padrão

Valores padrão permitem que um parâmetro tenha um valor pré-definido. Se quem chama a função não passar aquele argumento, o valor padrão é usado

automaticamente. Isso torna o parâmetro **opcional**.

```
def gerar_url_teste(endpoint, ambiente="dev"):
    bases = {
        "dev": "https://dev.api.com",
        "hml": "https://hml.api.com",
        "prod": "https://api.com"
    }
    base = bases.get(ambiente, "https://dev.api.com")
    return f"{base}/{endpoint}"

# Sem passar ambiente: usa "dev"
gerar_url_teste("users")          # https://dev.api.com/users

# Passando ambiente: sobrescreve o padrão
gerar_url_teste("users", "hml")   # https://hml.api.com/users
gerar_url_teste("users", "prod")  # https://api.com/users
```

Dica de QA: A maioria dos testes roda em dev, então faz sentido que esse seja o padrão. Para rodar em outro ambiente, basta passar o argumento. Sem valores padrão, você teria que escrever `"dev"` em todas as chamadas.

Com múltiplos valores padrão, o caso mais comum fica simples e os casos especiais ficam configuráveis:

```
def criar_usuario_teste(nome, perfil="viewer", ativo=True):
    return {"nome": nome, "perfil": perfil, "ativo": ativo}

criar_usuario_teste("Ana QA")          # perfil=viewer, ativo=True
criar_usuario_teste("Carlos", "admin") # perfil=admin, ativo=True
criar_usuario_teste("Bot", "api", False) # perfil=api, ativo=False
```

3. Regra de Posição dos Valores Padrão

O Python impõe uma regra: parâmetros **com** valor padrão devem vir **depois** dos parâmetros **sem** valor padrão.

```
# Correto
def func(obrigatorio, opcional="valor"):

# Errado -> SyntaxError
def func(opcional="valor", obrigatorio):
```

Pense assim: se o primeiro parâmetro tem padrão e o segundo não, como o Python saberia se o valor que você passou é para o primeiro ou para o segundo?

```
def configurar_teste(nome_teste, timeout=30, retries=3, verbose=False):
    return {
        "nome": nome_teste,
        "timeout": timeout,
        "retries": retries,
        "verbose": verbose
    }
```

```
configurar_teste("login_test")          # usa todos os padrões
configurar_teste("login_test", 60)      # muda só timeout
```

Posição	Tipo	Exemplo
Primeiro	Sem padrão (obrigatório)	<code>nome_teste</code>
Depois	Com padrão (opcional)	<code>timeout=30</code>

4. Argumentos Nomeados (Keyword Arguments)

Até agora, passamos argumentos pela **posição**: o primeiro valor vai para o primeiro parâmetro, o segundo para o segundo. Isso são **argumentos posicionais**.

Argumentos **nomeados** permitem especificar qual parâmetro recebe qual valor usando o nome:

```
def gerar_relatorio(suite, total, passou, falhou, ambiente="dev"):  
    taxa = (passou / total * 100) if total > 0 else 0  
    return f"[{ambiente.upper()}] {suite}: {passou}/{total} ({taxa:.0f}%)"  
  
# Posicional (pela ordem)  
gerar_relatorio("Login", 10, 8, 2)  
  
# Nomeado (pelo nome)  
gerar_relatorio(suite="Login", total=10, passou=8, falhou=2)
```

As duas grandes vantagens dos argumentos nomeados:

Pular parâmetros com valor padrão:

```
# Pulo direto para "ambiente" sem passar os do meio  
gerar_relatorio("Login", 10, 8, 2, ambiente="prod")
```

Mudar a ordem dos argumentos:

```
# A ordem não importa quando usa nomes  
gerar_relatorio(falhou=2, suite="Login", passou=8, total=10)
```

Dica de QA: Em testes, argumentos nomeados tornam o código muito mais legível. Quando você lê `gerar_relatorio(suite="Login", total=10, passou=8, falhou=2)`, fica claro o que cada valor significa sem precisar olhar a definição da função.

5. Combinando Posicionais e Nomeados

Na prática, você vai misturar os dois estilos. A regra é: **posicionais sempre antes dos nomeados**.

```
def executar_request(metodo, endpoint, timeout=30, headers=
None, auth=False):
    config = {
        "metodo": metodo,
        "endpoint": endpoint,
        "timeout": timeout,
        "headers": headers or {},
        "auth": auth
    }
    return config

# Obrigatórios por posição, opcionais por nome
executar_request("GET", "/api/users")
executar_request("POST", "/api/users", auth=True, timeout=6
0)
executar_request("GET", "/api/admin", headers={"X-Role": "a
dmin"}, auth=True)
```

Repare que no segundo exemplo, pulamos `headers` e fomos direto para `auth` e `timeout`. Sem argumentos nomeados, teríamos que passar `None` para headers manualmente.

```
# Correto
func(valor1, valor2, nome="x")

# Errado -> SyntaxError
func(nome="x", valor1, valor2)
```

Dica de QA: Esse padrão (posicional para obrigatórios, nomeado para opcionais) é o mais usado em bibliotecas como `requests` e `pytest`. Reconhecer esse padrão ajuda a entender qualquer biblioteca Python.

6. Cuidado com Valores Padrão Mutáveis

Essa é uma armadilha clássica do Python. Quando você usa uma lista ou dicionário como valor padrão, esse objeto é criado **uma única vez** na definição da função e **compartilhado** entre todas as chamadas.

```
# ERRADO: a lista é compartilhada entre chamadas
def adicionar_log_errado(mensagem, logs=[]):
    logs.append(mensagem)
    return logs

adicionar_log_errado("Erro 404") # ['Erro 404']
adicionar_log_errado("Erro 500") # ['Erro 404', 'Erro 500'] <- Acumulou!
```

A solução é usar `None` como padrão e criar o objeto dentro da função:

```
# CORRETO: cada chamada cria sua própria lista
def adicionar_log_correto(mensagem, logs=None):
    if logs is None:
        logs = []
    logs.append(mensagem)
    return logs

adicionar_log_correto("Erro 404") # ['Erro 404']
adicionar_log_correto("Erro 500") # ['Erro 500'] <- Independente!
```

Padrão	Exemplo	Resultado
Errado	<code>logs=[]</code>	Lista compartilhada entre chamadas
Correto	<code>logs=None</code>	Lista nova a cada chamada

Dica de QA: Essa é uma das perguntas mais comuns em entrevistas de Python. A regra é simples: nunca use lista, dicionário ou set como valor padrão. Sempre use `None` e crie o objeto dentro da função.

7. Aplicação Prática QA: Função de Validação Flexível

Cenário real: uma função que valida respostas de API com diferentes níveis de verificação.

```
def validar_resposta_api(
    status_code,
    corpo,
    status_esperado=200,
    campos_obrigatorios=None,
    tempo_resposta=None,
    max_tempo=5.0
):
    if campos_obrigatorios is None:
        campos_obrigatorios = []
    erros = []
    if status_code != status_esperado:
        erros.append(f"Status: esperado {status_esperado},
recebeu {status_code}")
    for campo in campos_obrigatorios:
        if campo not in corpo:
            erros.append(f"Campo ausente: '{campo}'")
    if tempo_resposta is not None and tempo_resposta > max_
tempo:
        erros.append(f"Tempo: {tempo_resposta}s excede limi
te de {max_tempo}s")
    return {"valido": len(erros) == 0, "erros": erros}
```

A mesma função serve para cenários completamente diferentes:

```
# Validação mínima (só status)
validar_resposta_api(200, {"id": 1})

# Com campos obrigatórios
validar_resposta_api(200, {"id": 1}, campos_obrigatorios=
["id", "nome"])

# Validação completa
validar_resposta_api(
    status_code=201,
    corpo={"id": 1},
    status_esperado=200,
    campos_obrigatorios=["id", "nome"],
```

```

        tempo_resposta=6.2
    )

# Endpoint de criação (status 201)
validar_resposta_api(201, {"id": 99}, status_esperado=201)

```

Essa flexibilidade toda vem dos valores padrão e argumentos nomeados.

Tabela Resumo

Conceito	O que é	Exemplo
Parâmetro	Variável na definição	<code>def func(x):</code>
Argumento	Valor na chamada	<code>func(10)</code>
Valor padrão	Torna parâmetro opcional	<code>def func(x=10):</code>
Arg. posicional	Atribuído pela ordem	<code>func(1, 2, 3)</code>
Arg. nomeado	Atribuído pelo nome	<code>func(x=1, y=2)</code>
Regra de posição	Obrigatórios antes dos opcionais	<code>def func(a, b=10):</code>
Regra de chamada	Posicionais antes dos nomeados	<code>func(1, b=10)</code>
Mutáveis	Usar None como padrão	<code>def func(lista=None):</code>